Day 146: Week 21 Build: a Text-to-Digit Generator, End to End

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 21, Day 7 | Complete study guide

Watch: How Stable Diffusion Works: AI Image Generation Explained (CodeLucky) | Coding Stable Diffusion from scratch in PyTorch (Umar Jamil)

Script: `day146_text_to_digit_capstone.py` · every number in this guide is copied by program from `day146_run_output.txt`, the real output of that script.

This week built a text-to-image system one part at a time: a noise schedule (Day 140), a network that removes noise (Day 141), a fast sampler (Day 142), a text encoder (Day 143), a text-conditioned model with guidance (Day 144) and checks for what it draws (Day 145). Today they become one generator: type a prompt, get handwritten digits. Putting them together found a real problem first: pasted into one file, the parts would silently damage Day 143's encoders (half their outputs set to exactly 0, 0.488). Imported instead, the system reproduced Day 143's and Day 144's numbers exactly. A small front end took '7' from 0.000 right to 1.000, 10 sampling steps cost about an eighth of 100 for nearly the same digits, and a model card shows what the generator does to each digit: every one thicker than real handwriting.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

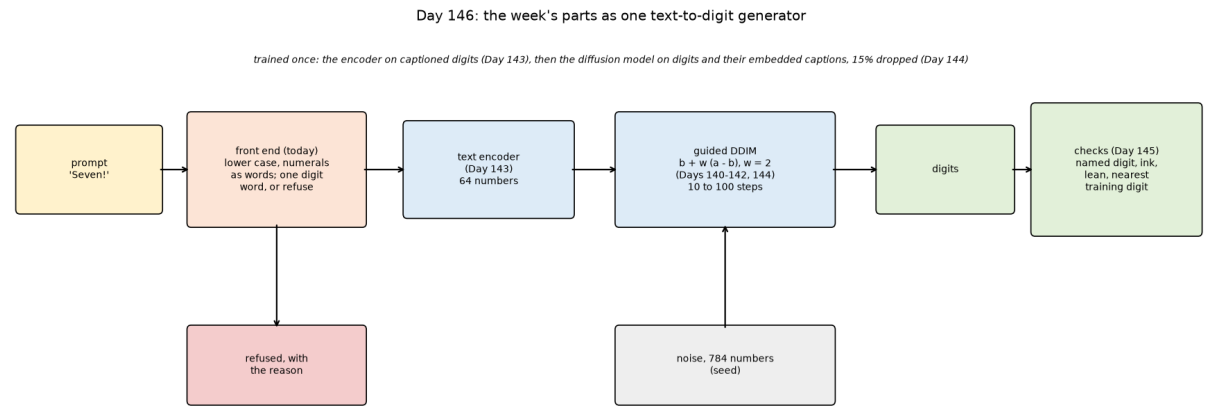
A **system** here means the trained parts joined so that a person can use them without knowing they exist: text goes in, images come out. **End to end** means every step from the typed prompt to the checked output is run, not just the model in the middle.

Joining parts raises questions no single part answers: do they still behave as they did alone, what happens to input the parts were never built for, how fast is a request, and what does the whole thing do to each kind of output?

REAL-WORLD SCENARIO

A team has a working image model in a research notebook and is asked to put it behind a text box on a website. The model is the easy part. They must make sure the shipped copy is the same model, decide what to do with prompts like 'Seven!' or 'a cat', pick the number of sampling steps that keeps the page fast, and write down what the model tends to get wrong. Those four jobs are sections 1–5 below, on digits.

SYSTEM DIAGRAM



The week's parts as one generator; only the front end is new code.

Data: Day 126's 70,000 MNIST digits (training images 0–59,999; 2,000 test digits in random order, as on Day 143). Day 138's classifier names generated digits and gives the scores (IS, FID, precision, recall). Every model is trained by an earlier day's own function with the same seeds: Day 143's encoders with the pair (seven, thick) held out, then Day 144's text-conditioned diffusion model. Today's build has no exercise file: it is the exercise files, imported.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. The parts: imported, not pasted

DEFINITION — start here, no prior knowledge assumed

A Python **module** is a file whose names (functions and constants) live in their own namespace: Day 143's functions, imported as `d143`, look up Day 143's constants. Pasting files together instead puts every name in one namespace, where a later assignment of the same name silently replaces an earlier one.

Earlier build days (125, 132, 139) pasted their week's parts. This week, two constant names are given different values in different files, so the check below reads every exercise file's constants with `ast` before anything is joined.

REAL-WORLD SCENARIO

Two departments each keep a document called 'price list'. Kept in separate folders, each department reads its own. Copied into one shared folder, the second copy overwrites the first, and the first department starts quoting the wrong prices without anyone seeing an error.

SPECIFICATION

Clash check	each of Days 140–145's exercise files parsed with ast; capitalised names assigned before the file's last function; a name is a clash if its written values differ
Damage	Day 143's encoders (2 layers) run with Day 143's ACTS and then with Day 144's 3-entry ACTS, which a pasted file would leave in force; zero-shot accuracy on the 2,000 test digits, and the share of embedding entries that are exactly 0

IMPLEMENTATION

```
def zero_shot_acts(img, txt, X, y, acts):
    T = d126.forward(txt, d143.bow([f"the digit {w}".split() for w in d143.NUM]),
        acts)[-1].astype(np.float64)
    Z = d126.forward(img, X, acts)[-1].astype(np.float64)
    zero = float(np.mean(np.vstack([T, Z]) == 0))
    T /= np.linalg.norm(T, axis=1, keepdims=True) + 1e-12; Z /= np.linalg.norm(Z, axis=1,
        keepdims=True) + 1e-12
    return float(((Z @ T.T).argmax(1) == y).mean()), zero
```

ACTUAL OUTPUT

```
constants given different values in this week's exercise files (in a single file, only the last would survive):
  SIZES: Day 141: [816, 512, 512, 784]; Day 144: [784 + 32 + 64, 512, 512, 784]
  ACTS: Day 141: ["relu", "relu", "linear"]; Day 143: ["relu", "linear"]; Day 144: ["relu", "relu", "linear"]
Day 143's encoders (2 layers) run with Day 143's ACTS ['relu', 'linear']: zero-shot accuracy 0.9785, embedding entries
    exactly 0: 0.000
the same encoders run with Day 144's ACTS ['relu', 'relu', 'linear'], as a pasted file would: zero-shot accuracy 0.9755,
    entries exactly 0: 0.488
no error either way: Day 126's forward pairs layers with activations using zip, which stops at the shorter list,
so the encoder's last layer quietly becomes a ReLU
Actual output from this lesson's run.
```

What it means

Two names clash. SIZES differs because Day 144's network takes the caption embedding as extra input; a wrong size would likely stop with a shape error (not tested here). ACTS fails silently. Day 126's forward pairs layers with activations using zip, which stops at the shorter list: Day 143's two-layer encoders run with Day 144's list ['relu', 'relu', 'linear'] get a ReLU on their last layer instead of a linear output. Nothing fails.

The damage is half hidden. Zero-shot accuracy barely moves (0.9785 to 0.9755), so a quick check would pass, but 0.488 of the embedding entries become exactly 0 (none did before). Day 144's diffusion model was trained on the undamaged embeddings, so in a pasted file it would be fed vectors unlike any it learned from; how much worse its digits would be was not measured here.

WHY CHOOSE THIS?

Importing keeps each part exactly as it was tested, so the earlier days' numbers still apply; a constant check costs a few lines.

WHY NOT THIS? (TRADEOFFS)

Imports tie today's script to the exercise files' locations and names; a single pasted file is easier to hand to someone else, which is why earlier build days did it.

2. The model, checked against Days 143 and 144

DEFINITION — start here, no prior knowledge assumed

A **regression check** reruns something that already worked and compares the new result with the recorded one. If the same code, data and seeds give a different number, something in the assembly changed the parts.

Here the recorded results are Day 143's and Day 144's own output files: the script reads the numbers from them and compares them with today's, character by character.

REAL-WORLD SCENARIO

Before a restaurant chain opens a new branch, a manager cooks the signature dish in the new kitchen and tastes it against the original. Same recipe and ingredients should give the same dish; if it does not, the kitchen, not the recipe, is wrong.

MATHEMATICS

$$\text{guided } x_0 = b + w\,(a - b)$$

a = the model's guess of the clean digit given the prompt, b = its guess with the prompt dropped, w = guidance scale (Day 144)

SPECIFICATION

Encoders	Day 143's train_clip on the 60,000 training digits, held-out pair (seven, thick), as on Day 144
Diffusion	Day 144's train_cond: Day 141's network with the caption embedding as extra input, 20,000 steps, 15% of captions dropped, EMA weights for sampling
Sampling	prompts 'zero' ... 'nine', 200 each, w = 2, Day 142's DDIM with 100 steps and no fresh noise, seed 1 + digit, as on Day 144

IMPLEMENTATION

```
old = next(ln for ln in open(os.path.join(HERE, "day144_run_output.txt"),
    encoding="utf-8").read().split("\n") if ln.startswith("  2.0 "))
print(f"prompts 'zero' ... 'nine', {N} samples each, w = 2, DDIM 100 steps, the same starting
    noise as Day 144:")
print(f"{'':>9}{'w':>5} {'right digit':>12} {'IS':>6} {'FID':>8} {'precision':>10} {'recall':>7}
    {'grey share':>11} {'ink':>6}")
print(f"{'today':>9}{row}   ({took:.0f} s)")
print(f"{'Day 144':>9}{old[:len(row)]}")
print(f"every printed digit the same as Day 144's run: {row == old[:len(row)]}")
```

ACTUAL OUTPUT

```
text encoder (Day 143's train_clip, the held-out pair): zero-shot accuracy 0.9785; Day 143's run printed 0.9785: the same
text-conditioned diffusion model (Day 144's train_cond, 20,000 steps): loss 0.1620 -> 0.1320, 886 s
prompts 'zero' ... 'nine', 200 samples each, w = 2, DDIM 100 steps, the same starting noise as Day 144:
            w  right digit     IS      FID  precision  recall  grey share    ink
    today  2.0       1.000   9.90    25.06      0.930   0.464      0.0848  0.150   (19 s)
  Day 144  2.0       1.000   9.90    25.06      0.930   0.464      0.0848  0.150
every printed digit the same as Day 144's run: True
Actual output from this lesson's run.
```

What it means

Every check passed. The encoders' zero-shot accuracy is Day 143's 0.9785, the loss runs 0.1620 -> 0.1320 as on Day 144, and the whole w = 2 row (right digit 1.000, IS 9.90, FID 25.06, recall 0.464) matches Day 144's printed row exactly: 'every printed digit the same as Day 144's run: True'.

Only the time differs: training took 886 s today against 559 s on Day 144, the same code on the same machine. Timings here swing by two times or more from run to run, so they are compared only within one run.

WHY CHOOSE THIS?

An exact match with a recorded run is strong evidence that the assembly changed nothing; it costs one extra comparison per number.

WHY NOT THIS? (TRADEOFFS)

It only works when everything is seeded and deterministic, and it checks that the parts agree with their past, not that they are right.

3. The front end: prompts as the encoder can read them, and refusals

DEFINITION — start here, no prior knowledge assumed

The text encoder knows a fixed list of lower-case words. A **front end** sits before it: it **normalises** the prompt (lower case, punctuation removed, a numeral like '7' turned into 'seven') and **refuses** a request it cannot serve, saying why, instead of drawing something anyway.

Here a request is served only if exactly one digit word is left after normalising; other unknown words are dropped and listed.

REAL-WORLD SCENARIO

A bank's phone line asks for an account number. It accepts '12 34' and '1234' alike, and if a caller says 'my savings account', it asks again rather than guessing which account they meant.

SPECIFICATION

Raw	the prompt split on spaces and embedded as typed (as on Day 144): unknown words, such as '7' or 'Seven!', are simply missing from the encoder's bag of words
Front end	normalise, then refuse unless exactly one digit word is left; known words such as 'thick' or 'leaning' are kept
Measured	200 samples per prompt, w = 2, the same starting noise; 'right': named as the intended digit by Day 138's classifier

IMPLEMENTATION

```
def normalise(prompt):
    words = re.findall(r"[a-z0-9]+", prompt.lower())
    words = [d143.NUM[int(w)] if len(w) == 1 and w.isdigit() else w for w in words]
    return [w for w in words if w in d143.IDX], [w for w in words if w not in d143.IDX]
def generate(txt, ema, prompt, n=N, w=W_GUIDE, negative=None, steps=100, seed=1):
    known, unknown = normalise(prompt)
    digits = sorted({k for k in known if k in d143.NUM}, key=d143.NUM.index)
    if len(digits) != 1:
        return None, "refused: " + ("no digit word it knows" if not digits else "two digits (" + "
            and ".join(digits) + ")")
    c = d143.embed_texts(txt, [known])[0]
    cn = None if negative is None else d143.embed_texts(txt, [normalise(negative)[0]])[0]
    note = "drawn" + (f"; ignored {', '.join(repr(u) for u in unknown)}" if unknown else "")
    return d144.cfg_sample(ema, c, n, w, cn, steps, seed), note
```

ACTUAL OUTPUT

```
200 samples per prompt, w = 2, the same starting noise (seed 1); 'raw': the prompt split on spaces and embedded as typed,
as on Day 144; 'front end': normalise, then refuse unless exactly one digit word is left; 'right': named as the intended
    digit
                prompt  raw: right          raw: named as   front end                                right
               'seven'       1.000      seven 200, zero 0   drawn                                    1.000
                   '7'       0.000      eight 84, five 24   drawn                                    1.000
              'Seven!'       0.000      eight 84, five 24   drawn                                    1.000
               'SEVEN'       0.000      eight 84, five 24   drawn                                    1.000
           'a thick 3'       0.025     eight 128, zero 28   drawn                                    1.000
      '4 leaning right'       0.510      four 102, nine 36   drawn                                    1.000
   'draw me a 9 please'       0.040     eight 115, five 29   drawn; ignored 'draw', 'me', 'please'     0.995
               'sept'       0.000      eight 84, five 24   refused: no digit word it knows               -
   'a picture of a cat'           -     eight 115, five 29   refused: no digit word it knows               -
         'three seven'           -    three 112, seven 86   refused: two digits (three and seven)         -
Actual output from this lesson's run.
```

What it means

Typed raw, every prompt the encoder could not read drew the same thing: '7', 'Seven!', 'SEVEN' and 'sept' all gave eight 84, five 24 and none was right (0.000), because none of their words is in the encoder's list. Through the front end the first three drew sevens every time (1.000), and 'sept' was refused.

Partly readable prompts drew from what was left. Raw 'a thick 3' kept only 'a' and 'thick' and drew mostly eights (eight 128, zero 28); raw '4 leaning right' drew fours 0.510 of the time from 'leaning right' alone, although every digit has a third of its captions leaning right; why the words favour fours was not tested here. Through the front end both were right 1.000, and 'draw me a 9 please' 0.995, with the ignored words listed.

Raw 'three seven' drew a mixture (three 112, seven 86): a request with two answers got a bit of each. The front end refuses it, and 'a picture of a cat', with the reason.

WHY CHOOSE THIS?

A few lines of normalising turned unreadable prompts into correct digits, and refusing says why instead of drawing a confident wrong answer.

WHY NOT THIS? (TRADEOFFS)

Hand-written rules only cover what their author thought of: 'sept' (French) and 'seventh' are still refused, and a real system with a large vocabulary needs a learned encoder, not rules.

4. Speed: how many sampling steps a request needs

DEFINITION — start here, no prior knowledge assumed

Each DDIM step runs the network twice with guidance (once with the prompt, once without), so the cost of a request grows in proportion to the number of steps. Day 142 found that, for an unconditioned model, fewer steps lowered quality.

REAL-WORLD SCENARIO

A printer has draft and best quality settings. Best is slower; whether it is worth it depends on whether anyone can see the difference on the page.

MATHEMATICS

$$\text{network calls per request} = 2 \times \text{steps} \times \text{samples}$$

$$\text{time per digit} \approx \text{constant} \times \text{steps (measured below)}$$

SPECIFICATION

Steps	10, 25, 50, 100 (100 is section 2's run, reused)
Kept fixed	the model, w = 2, the starting noise, 200 samples per prompt

IMPLEMENTATION

```
for s in STEP_LIST:
    if s == 100:
        F, took = F100, t100
    else:
        t0 = time.perf_counter(); F = np.vstack([d144.cfg_sample(ema, d143.embed_texts(txt,
            [[w]])[0], N, W_GUIDE, steps=s, seed=1 + k) for k, w in enumerate(d143.NUM)])
        took = time.perf_counter() - t0
    _, fid, _, r, _ = sc(F); acc = float(np.mean(d144.said(sc, F) == yy)); rows.append((s, acc,
        fid, took)); keep[s] = F[::N // 4][:40]
    print(f"{s:6d} {acc:12.3f} {fid:8.2f} {r:7.3f} {d142.grey_share(F):11.4f} {took:6.1f}s {1000 *
        took / len(F):13.2f}")
```

ACTUAL OUTPUT

```
prompts 'zero' ... 'nine', 200 samples each, w = 2 (two network calls per step), DDIM with no fresh noise, the same starting
    noise;
time for all 2,000 samples on this machine (timings vary from run to run here; compare within this table)
 steps  right digit      FID  recall  grey share    time  ms per digit
    10        1.000    24.02   0.342      0.1004    2.3s          1.14
    25        1.000    22.46   0.418      0.0924    5.0s          2.48
    50        1.000    23.47   0.450      0.0880   10.6s          5.29
   100        1.000    25.06   0.464      0.0848   19.4s          9.70
Actual output from this lesson's run.
```

What it means

Time grew with the steps, as the formula says: 1.14 ms per digit at 10 steps, 9.70 at 100. Every setting drew the right digit 1.000 of the time.

Quality did not fall the way Day 142's did. FID was 24.02 at 10 steps and 25.06 at 100, lowest at 25 (22.46); Day 142's unconditioned model went from 45.8 at 100 steps to 76.4 at 10. What fewer steps did cost was variety: recall fell from 0.464 to 0.342. A likely reason, not tested here: guidance and a text prompt pin down most of the digit early, so the remaining steps mostly add variety rather than shape.

The two grids in the figure below (10 and 100 steps, same starting noise) are hard to tell apart.

WHY CHOOSE THIS?

For this model, 10 to 25 steps give the right digit at about a quarter or less of the cost of 100, with FID as good.

WHY NOT THIS? (TRADEOFFS)

Fewer steps lowered recall, so the output is less varied; and a 'best' setting must be re-measured for each model, since Day 142's model behaved differently.

5. A model card: what this generator does to each digit

DEFINITION — start here, no prior knowledge assumed

A **model card** is a short document shipped with a model that reports how it behaves for each kind of input and where it fails (Mitchell and colleagues, 'Model Cards for Model Reporting', 2019). Here it is one row per prompt, using this week's checks: is the digit right, how much ink, how thick and leaning (Day 143's thirds), and how close to a training digit (Day 145's near-copy test).

REAL-WORLD SCENARIO

A food label lists what is in the product per serving, so a buyer can decide without testing it themselves. A model card does the same for a model's behaviour.

SPECIFICATION

Samples	section 2's: 200 per prompt, w = 2, 100 steps
Thick, thin, lean	Day 143's cut points for that digit's training images, so real training digits score a third in each
Nearest	median pixel distance to the nearest of the 60,000 training digits; the same for real test digits of the same kind; near copy: closer than 2.0 (Day 145)

IMPLEMENTATION

```
for d in range(10):
    k = slice(d * N, (d + 1) * N); tr = y[:60000] == d
    a1, a2 = np.percentile(ink[:60000][tr], [100 / 3, 200 / 3]); b1, b2 =
        np.percentile(slant[:60000][tr], [100 / 3, 200 / 3])
    c = (np.mean(lab[k] == d), ik[k].mean(), ink[:60000][tr].mean(), np.mean(ik[k] > a2),
        np.mean(ik[k] < a1), np.mean(sl[k] > b2),
         np.mean(sl[k] < b1), np.median(dF[k]), np.median(dT[y[te] == d]), np.mean(dF[k] < NEAR));
            card.append(c)
```

ACTUAL OUTPUT

```
section 2's samples (200 per prompt, w = 2); 'thick', 'thin', 'lean R' and 'lean L' use Day 143's cut points for that digit,
so real training digits score a third on each; ink: the average pixel; nearest: median pixel distance to the nearest
training digit, for the samples and for real test digits of the same kind; near copies: closer than 2.0
named: the share Day 138's classifier names as the prompted digit; lean R / lean L: leaning right / left
 prompt  named    ink   real  thick   thin lean R lean L  nearest   test  near copies
   zero  1.000  0.197  0.173  0.580  0.045  0.215  0.250     3.64   4.37        0.005
    one  1.000  0.083  0.076  0.570  0.010  0.150  0.140     1.66   1.67        0.900
    two  1.000  0.177  0.149  0.750  0.030  0.305  0.215     4.69   5.01        0.000
  three  1.000  0.162  0.142  0.590  0.075  0.265  0.220     3.99   4.71        0.000
   four  1.000  0.137  0.121  0.640  0.015  0.215  0.205     3.71   4.30        0.000
   five  1.000  0.148  0.129  0.645  0.055  0.395  0.115     4.25   4.85        0.000
    six  1.000  0.148  0.137  0.450  0.100  0.150  0.225     3.43   4.16        0.000
  seven  0.995  0.147  0.115  0.755  0.045  0.225  0.300     2.99   3.68        0.015
  eight  1.000  0.171  0.150  0.685  0.040  0.270  0.130     3.99   4.98        0.000
   nine  1.000  0.129  0.123  0.365  0.095  0.315  0.190     3.00   3.89        0.000
    all  1.000  0.150  0.131  0.603  0.051  0.251  0.199     3.64   4.32        0.092
closest sample to any training digit: 1.22; real test digits closer than 2.0: 0.090
Actual output from this lesson's run.
```

What it means

Every prompt drew its digit (0.995 for seven, 1.000 for the rest), but not the way people write them. Every digit had more ink than the real ones (all: 0.150 against 0.131), 0.603 were in the thickest third and only 0.051 in the thinnest. Fewer leaned either way than in real handwriting (0.251 right, 0.199 left, against a third each). Day 144's guidance table showed ink rising with w (0.124 at w = 0, 0.150 at w = 2, real 0.132), so guidance is a likely cause; the card itself only shows the effect.

Near copies look alarming for ones (0.900) and are not. Real test ones, which the model never saw, have a median distance of 1.67 to their nearest training one, so more than half of them would also count as 'near copies'; the samples' 1.66 is the same. Ones are simple strokes, and 2.0 is too loose a threshold for them. Across all digits, near copies were 0.092 against 0.090 for real test digits.

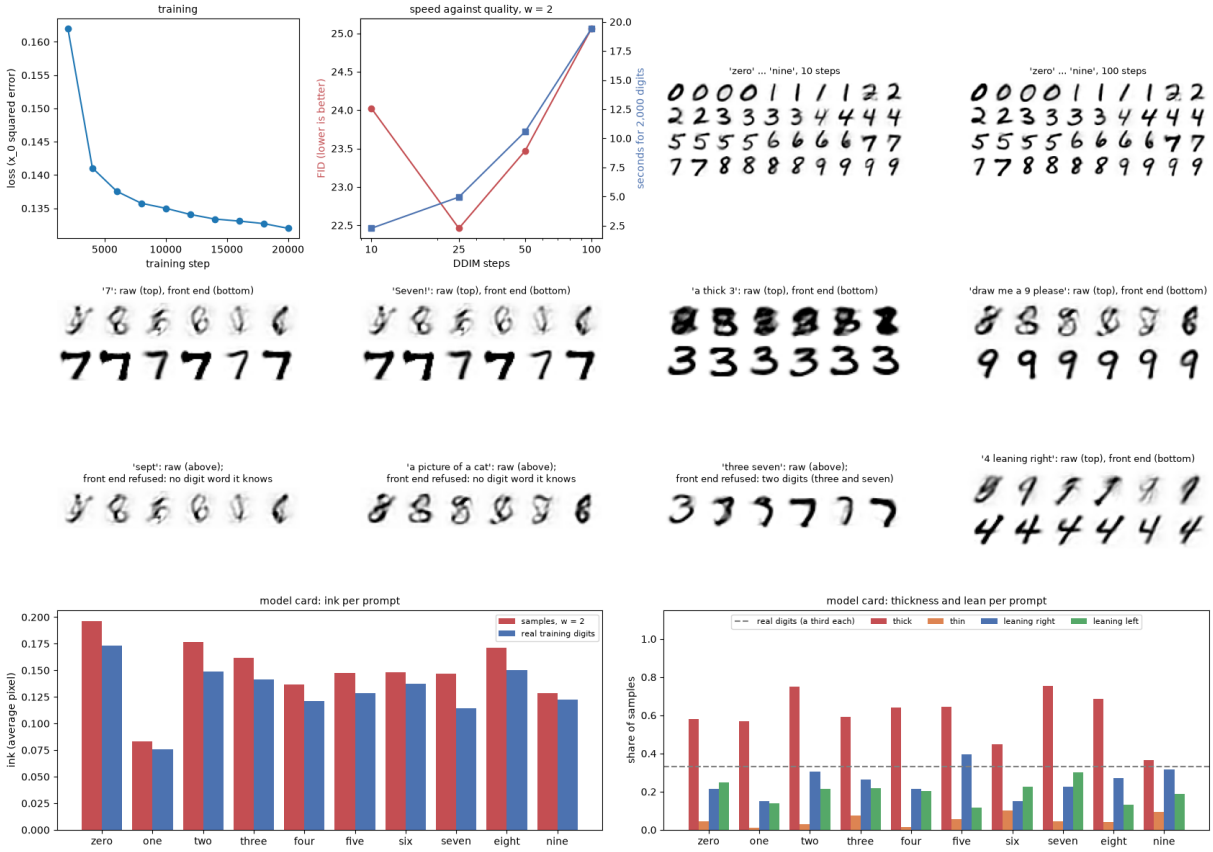
The samples did sit closer to the training set than real new digits do (median 3.64 against 4.32), the same pattern Day 145 found with 1,000 training digits.

WHY CHOOSE THIS?

A per-prompt table finds skews an overall score hides: FID was good while every digit was drawn too thick.

WHY NOT THIS? (TRADEOFFS)

Each column needs a reference to compare with (real digits, test distances), and a single threshold such as 2.0 can mislead for some kinds of input, as the ones show.



Top: training loss; FID and time against DDIM steps; 'zero' ... 'nine' at 10 and at 100 steps from the same noise. Middle: for eight prompts, six raw samples above six from the front end (refused prompts show only the raw row). Bottom: the model card, ink per prompt and the shares thick, thin and leaning against a third.

6. The week in one table

Day	Part	Used today
140	noise schedule, forward diffusion	inside Day 144's training and sampling
141	denoising network (x_0 output), nearest training digit	the network; the model card's distances
142	DDIM sampler, grey share	10–100 steps in section 4; FID at 10 steps 24.02
143	text and image encoders, zero-shot	the prompt's embedding; zero-shot 0.9785, as on Day 143
144	text-conditioned model, classifier-free guidance	the generator; w = 2 row identical (FID 25.06)
145	bias, detection, memorisation checks	the model card; near copies 0.092 (test 0.090)
146	front end, clash check (new today)	'7' right 0.000 raw, 1.000 with the front end

Interview preparation — technical questions

Q: How would you check that an assembled ML system still matches its parts?

A: Rerun with the same seeds and compare with recorded outputs number by number; here zero-shot 0.9785 and the whole w = 2 row matched exactly.

Q: Why might a model give confident wrong output for a prompt like '7'?

A: Its encoder did not know the word, so the prompt was effectively empty; here raw '7' drew eights 84 of 200 times and was right 0.000.

Q: How do you choose the number of diffusion sampling steps?

A: Measure quality, variety and time at several settings on the real model; here 10 steps gave FID 24.02 at 1.14 ms per digit against 25.06 at 9.70 for 100, but lower recall.

Q: What belongs in a model card for an image generator?

A: Per-prompt behaviour against a reference: right rate, attributes (here 0.603 thick against a third) and memorisation, with the reference for each.

Q: How can a missing error be worse than an error?

A: Day 126's forward zips layers with activations, so a wrong ACTS list ran silently and zeroed 0.488 of the embedding entries.

Interview preparation — theoretical questions

Q: Why did fewer steps hurt today's model less than Day 142's?

A: Measured: FID 24.02 at 10 steps here against 76.4 for Day 142's model. A likely reason, not tested here: the prompt and guidance fix most of the digit early.

Q: Why does guidance make digits thicker?

A: Measured on Day 144: ink rose from 0.124 at w = 0 to 0.150 at w = 2. A likely reason, not tested here: pushing away from the unprompted guess exaggerates the strokes typical of the digit.

Q: Is refusing a request better than drawing something?

A: For a request with no single answer, yes here: raw 'three seven' drew a mix (three 112, seven 86) that matches neither. Whether users prefer it was not tested.

Q: Why are 'near copies' of ones not evidence of memorisation?

A: Real test ones were as close (median 1.67) as the samples (1.66); a fixed threshold ignores how close real new data already is.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> The week's parts worked together only after one check that no single day needed: two files gave the same constant different values, and pasting them would have damaged the encoders without an error. Imported, the system reproduced Days 143 and 144 exactly. Around the model, a few lines of front end fixed messy prompts without touching the model, 10 steps were about as good as 100 for an eighth of the time, and the model card found what the scores hid: every digit drawn too thick.

- Clash: Day 143's encoders with Day 144's ACTS kept zero-shot 0.9755 but zeroed 0.488 of entries.
- Regression: zero-shot 0.9785 and the w = 2 row (FID 25.06) identical to Days 143 and 144.
- Front end: '7' right 0.000 raw, 1.000 normalised; 'three seven' and 'a picture of a cat' refused.
- Speed: 1.14 ms per digit at 10 steps, 9.70 at 100; FID 24.02 against 25.06.
- Model card: thick 0.603, thin 0.051 against a third each; near copies 0.092 against 0.090 for real test digits.

Suggested watch order, if starting from zero

- 1. How Stable Diffusion Works: AI Image Generation Explained (CodeLucky) — how a full text-to-image system fits together, the same parts at a larger scale.
- 2. Coding Stable Diffusion from scratch in PyTorch (Umar Jamil) — the same system built in code, to compare with this week's from-scratch parts.

Day 147 preview

- **Week 22 begins: time series.** Day 147 decomposes a real series into trend, seasonality and what is left over (the residual), the first step before forecasting it.